Architecture Transformer

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyb er Sécurité



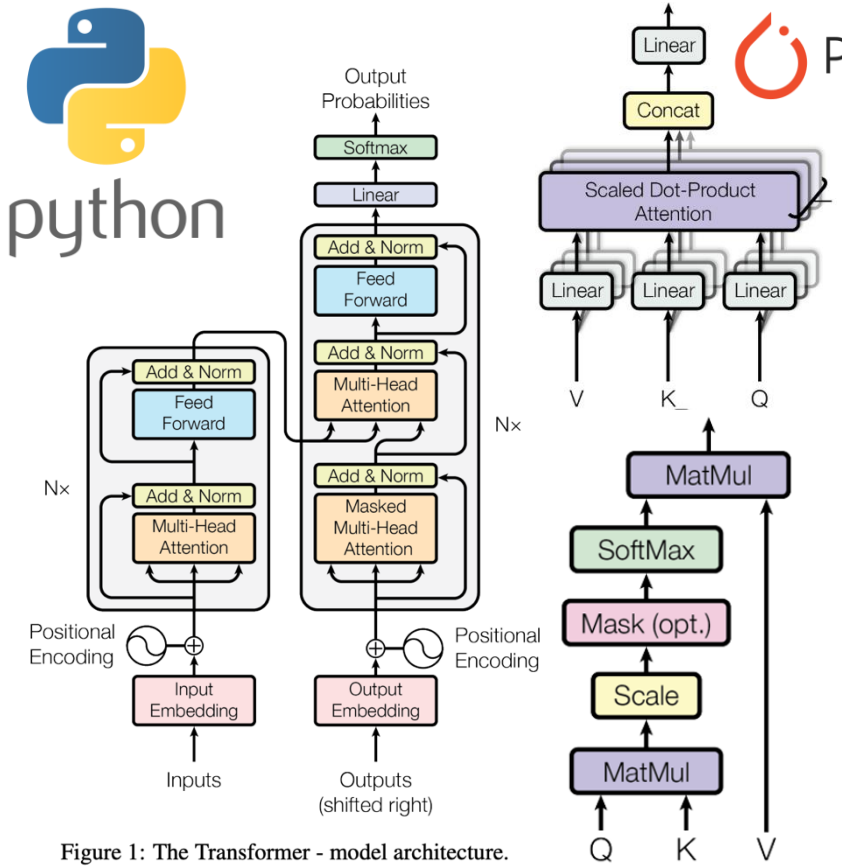
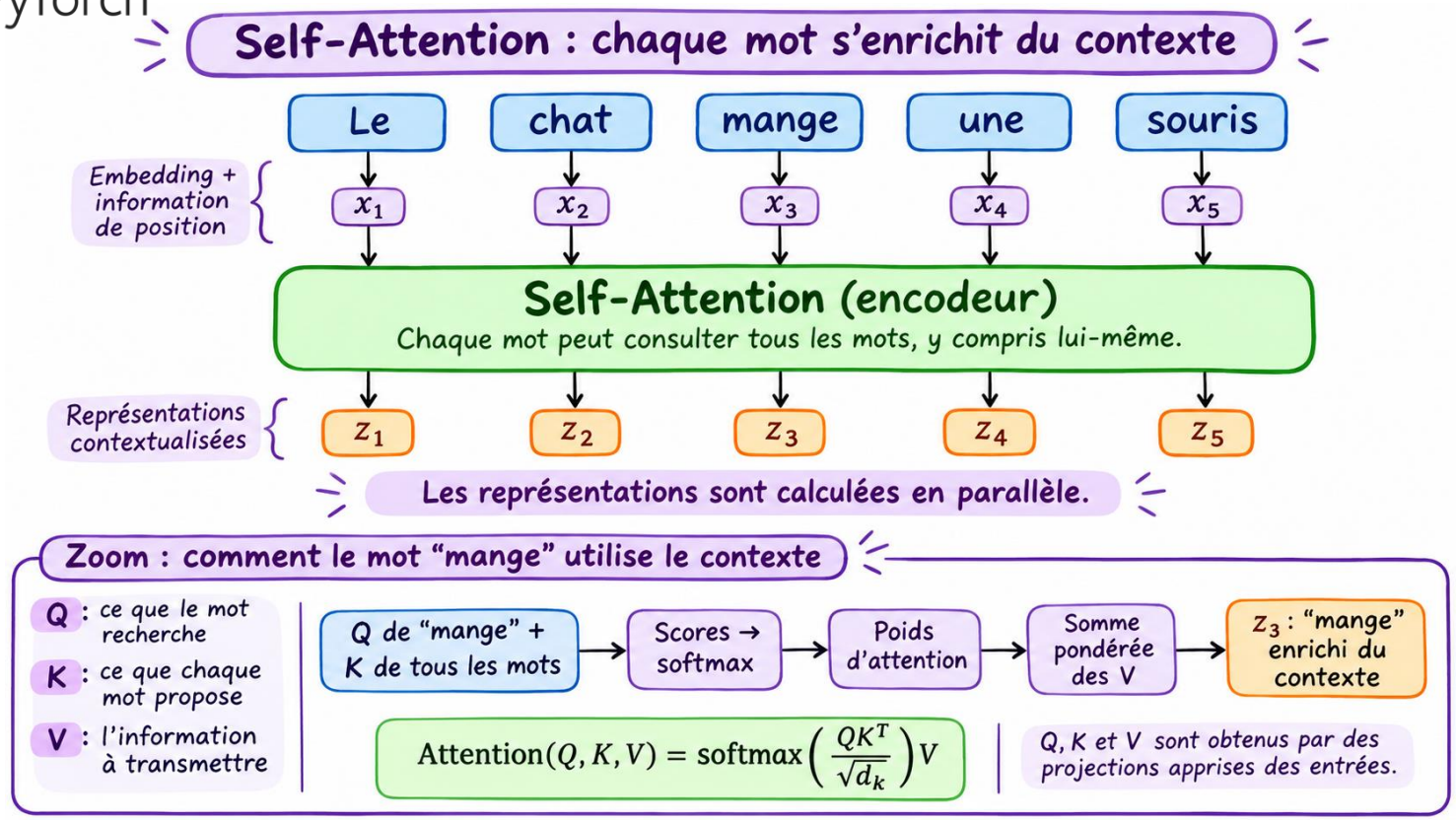


Figure 1: The Transformer - model architecture.



Generative AI (State Of Art)

- Generative AI refers to a type of Artificial Intelligence that is capable of generating new data, such as images, music, or text, based on a set of training data.
- Reasons that have led to a significant acceleration in the pace of research and development in generative AI.
  - **Powerful Language Models** :
    - For NLP : (GPT, BERT) : Machine translation, Text summarization, Sentiment analysis, Conversational AI
    - For Computer Vision : GANs, Stable Diffusion, DALL-E, Midjourney
  - **Abundant Data**:
    - In 2022, users sent around **650 million Tweets** per **day**.
    - In 2022, **333.2 billion emails** were sent every **day**.
  - **Hardware Advancements** : **GPU & TPU , Cloud Computing**

https://thispersondoesnotexist.com/

2014
- Generative Adversarial Networks (GANs) (Ian Goodfellow)

2017
- Break through in NLP : **Transformers** (google)

Attention is All You need (https://arxiv.org/pdf/1706.03762.pdf)

2018
- Advent of Large Models :
  - **BERT** (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers) , Google
  - **GPT** (**G**enerative **P**re-trained **T**ransformers **) (Open AI)**

2019
- Advanced GANs – Style GANs

2020
- **GPT-3** (Premier LLM)(Open AI)

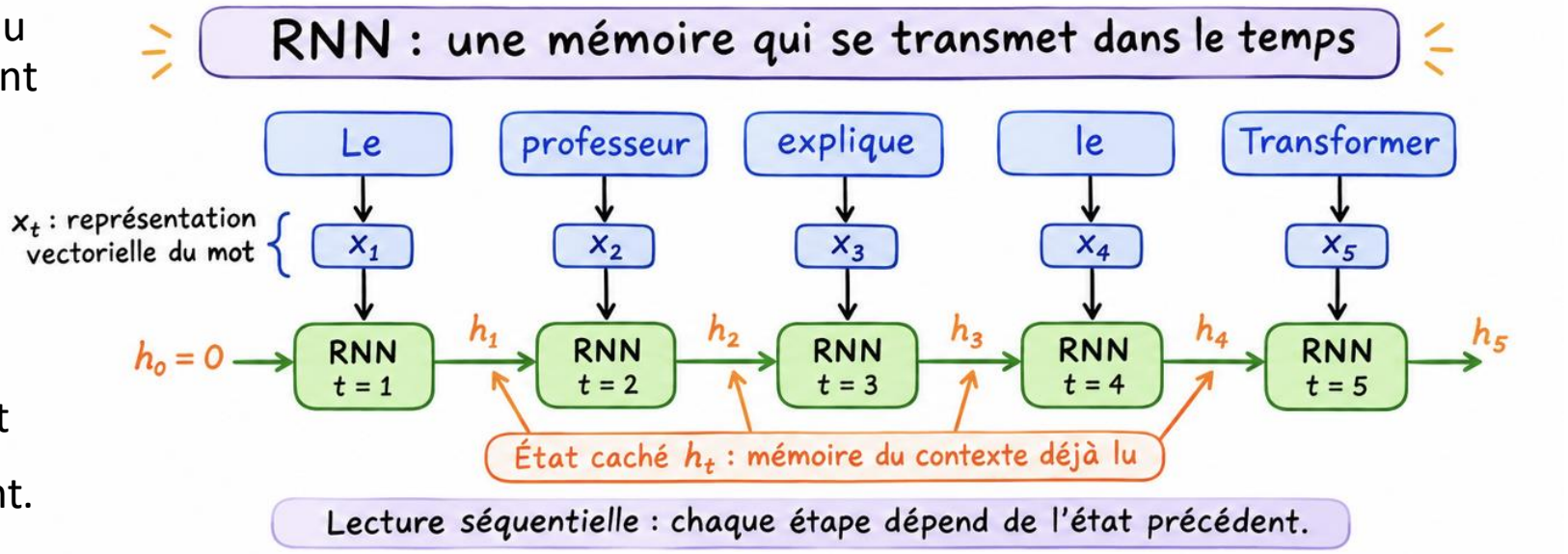
2021
- Vision Transformers – CLIP, **DALL-E** : Images (OpenAI)

2022
- Chat GPT : **GPT – 3.5 (Text)**, **DALL-E 2(Images), Stable Diffusion**

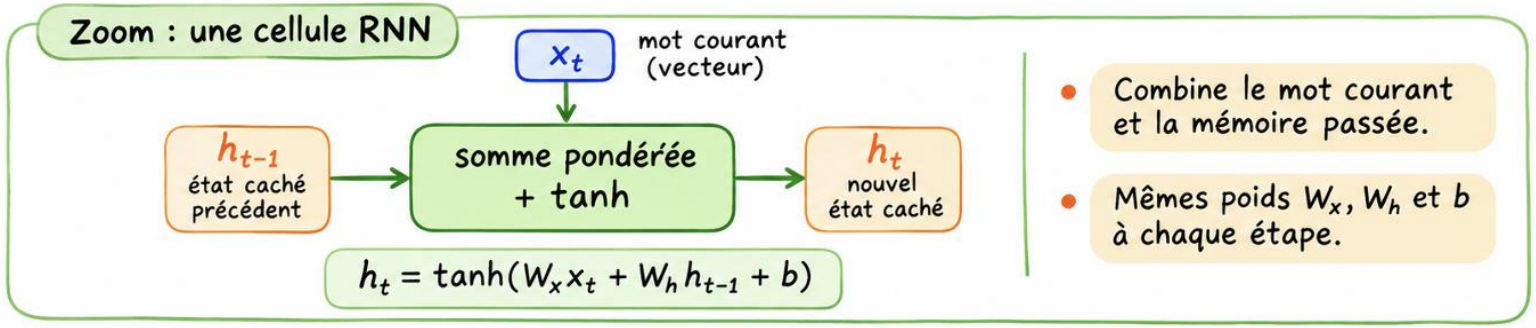
.. 26
- GPT-4o, GPT5, GPT6, Claude 3, Claude 4, GMINI 3, LIAMMA, ….
- IA Générative Multimodale et Agentique,

Avant 2017

- Avant 2017, les modèles de traitement du langage naturel reposaient principalement sur les RNN, LSTM et GRU.
- Exemple de texte: « **Le professeur explique le fonctionnement du Transformer aux étudiants.** »



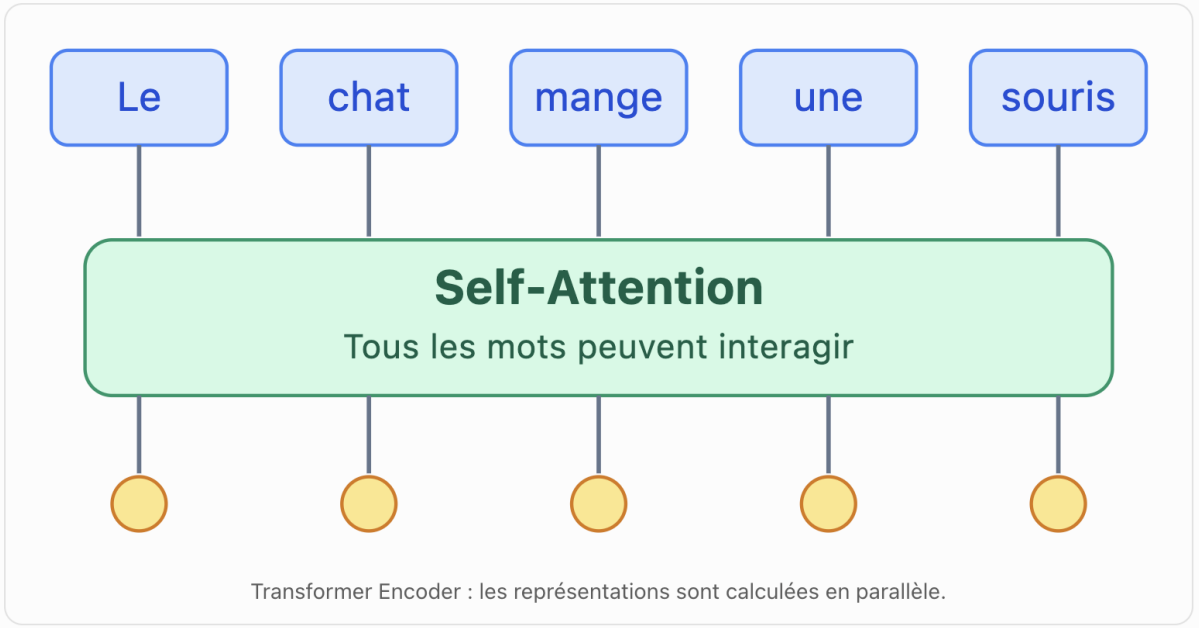
- Un RNN traite les mots séquentiellement
- Chaque étape dépend de l'état précédent.
- Cette approche présente trois limites principales :
  - **Traitement séquentiel** : difficile de paralléliser les calculs.
  - **Dépendances longues** : les informations éloignées peuvent se perdre.
  - **Vanishing Gradient** : le gradient peut devenir très faible lors de la rétropropagation.
- Les **LSTM** et **GRU** réduisent certaines de ces difficultés, mais conservent le traitement récurrent de manière séquentielle



$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

L'idée révolutionnaire du Transformer

- En 2017, l'article Attention Is All You Need introduit une architecture qui repose principalement sur un mécanisme :
  - **Attention Mechanism**.
- **Au lieu** de traiter obligatoirement les mots **les uns après les autres**, le Transformer peut analyser **simultanément** les relations entre les différents mots d'une séquence.
- L'avantage est que le modèle peut apprendre que, dans une phrase, deux mots éloignés ont une relation importante.

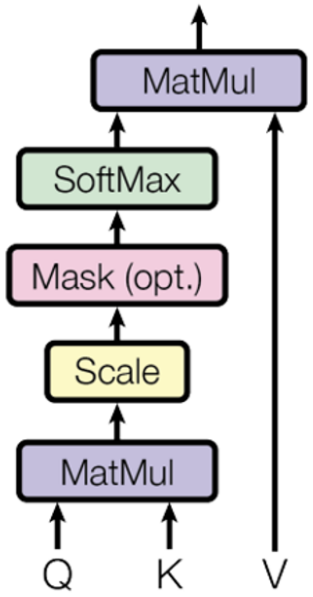


Transformer Encoder : les représentations sont calculées en parallèle.

https://arxiv.org/pdf/1706.03762.pdf



Mécanisme d'attention



Self-Attention : chaque mot s'enrichit du contexte

Le | chat | mange | une | souris

Embedding + information de position

x_1 x_2 x_3 x_4 x_5

Self-Attention (encodeur)

Chaque mot peut consulter tous les mots, y compris lui-même.

Représentations contextualisées

z_1 z_2 z_3 z_4 z_5

Les représentations sont calculées en parallèle.

Zoom : comment le mot "mange" utilise le contexte

- **Q** : ce que le mot recherche
- **K** : ce que chaque mot propose
- **V** : l'information à transmettre

Q de "mange" + K de tous les mots → Scores → softmax → Poids d'attention → Somme pondérée des V → z_3 : "mange" enrichi du contexte

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q, K et V sont obtenus par des projections apprises des entrées.

Chaque mot cherche à savoir quels autres mots sont importants pour comprendre son propre sens.

Mécanisme d’attention

- **Un exemple encore plus intéressant : un mot ambigu**

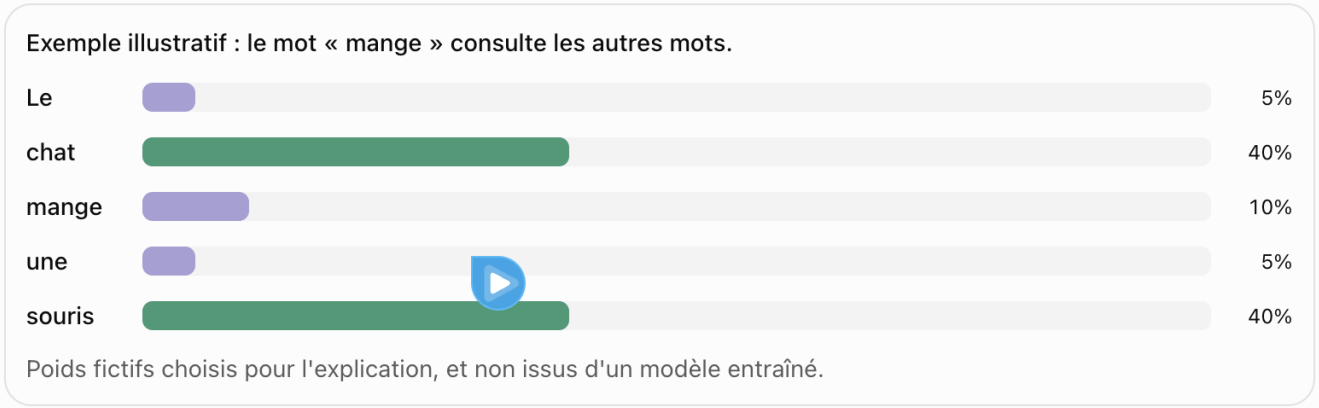
Prenons deux phrases :

  - « La **souris** mange du fromage. »
  - « La **souris** est connectée à l'ordinateur. »

- Le mot souris est identique, mais sa signification dépend du contexte.
- **Dans la première phrase**, le mot « souris » peut accorder davantage d'attention à « mange » et « fromage ».
- **Dans la deuxième phrase**, il peut accorder davantage d'attention à « connectée » et « ordinateur ».
- Ainsi, le même mot peut obtenir deux représentations contextualisées différentes, grâce à l'attention.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

En simplifiant, voici ce qui se passe :



Ici, le mot « mange » accorde 40 % d'attention à « chat » et 40 % à « souris ».

Mais attention à une subtilité importante : **ce ne sont pas les mots eux-mêmes qui sont additionnés.** Ce sont leurs vecteurs V qui sont combinés selon les poids calculés.

La distinction essentielle à retenir :

- Q : avec quelles informations ce token cherche-t-il à entrer en relation ?
- K : dans quelle mesure ce token correspond-il à cette recherche ?
- V : quelles informations numériques ce token transmet-il une fois son importance calculée ?

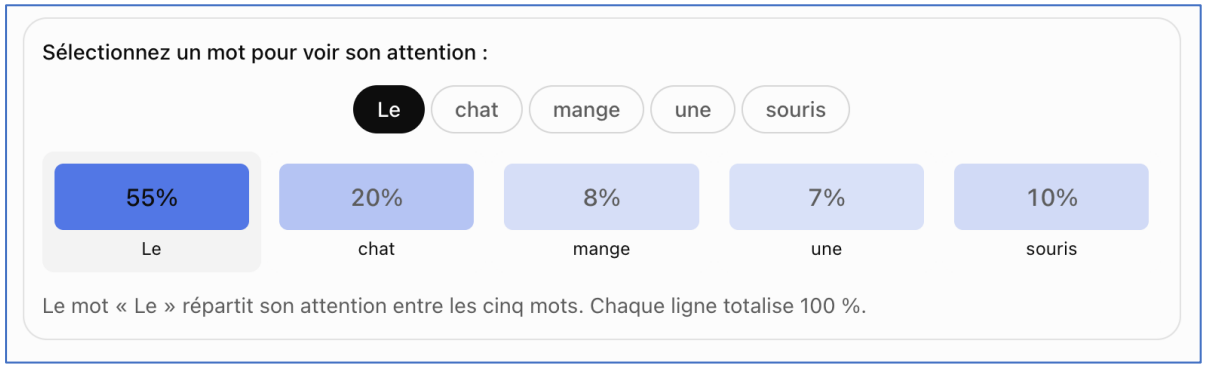
Et surtout, chaque token possède ses propres Q, K et V. Dans la Self-Attention, chaque mot peut donc être à la fois celui qui consulte et celui qui fournit des informations aux autres.

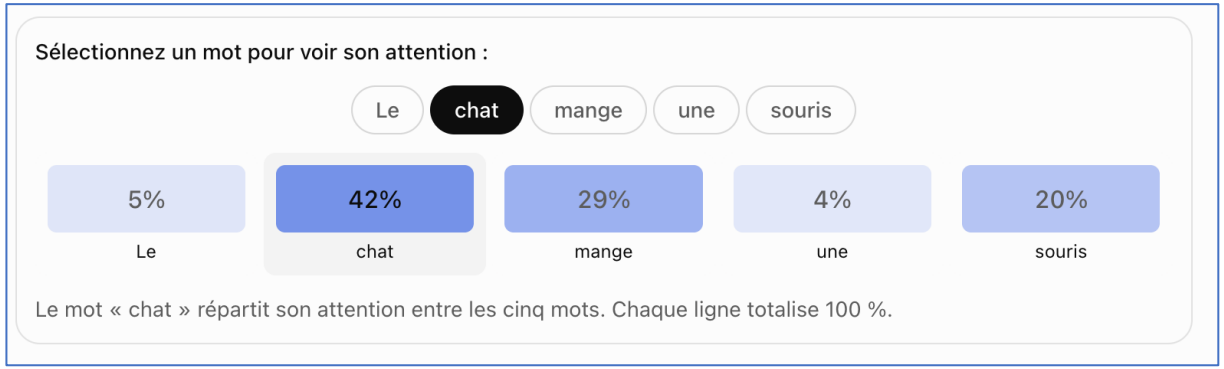
Mécanisme d'attention

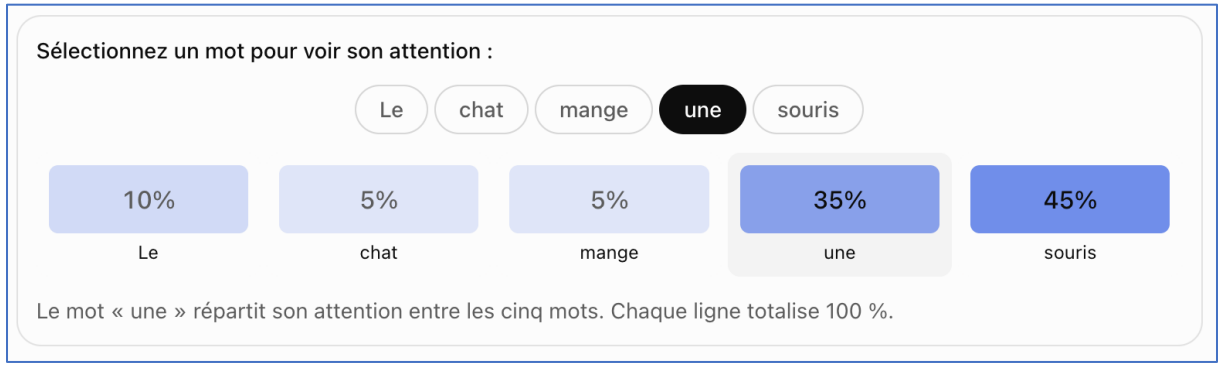
- C'est le cœur de l'architecture.
- Pour chaque token, le Transformer calcule trois vecteurs :
  - **Query (Q)** : ce que le token cherche.
  - **Key (K)** : ce que le token propose pour être retrouvé.
  - **Value (V)** : l'information qu'il apporte.
- Le calcul de l'attention s'écrit :

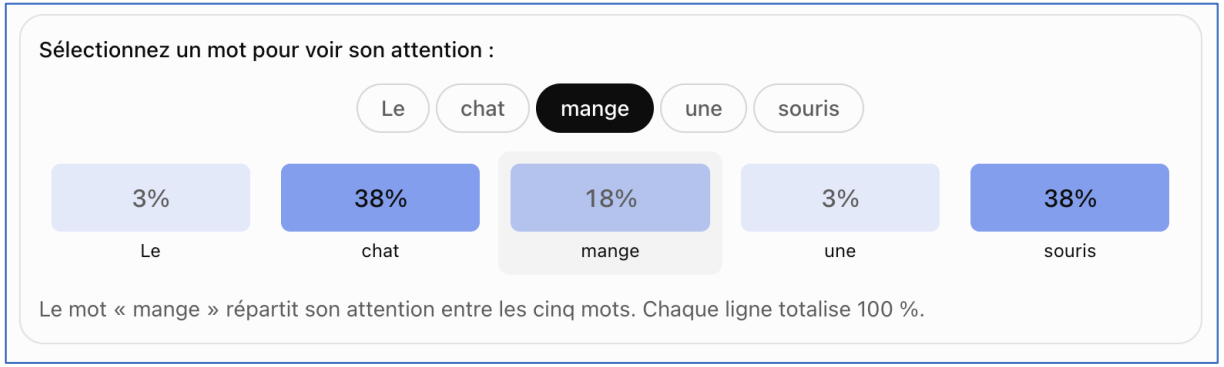
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Le produit QK^T calcule les scores de compatibilité entre les tokens.
- La division par $\sqrt{d_k}$ évite des scores trop grands
- **`softmax`** transforme les scores en poids d'attention.





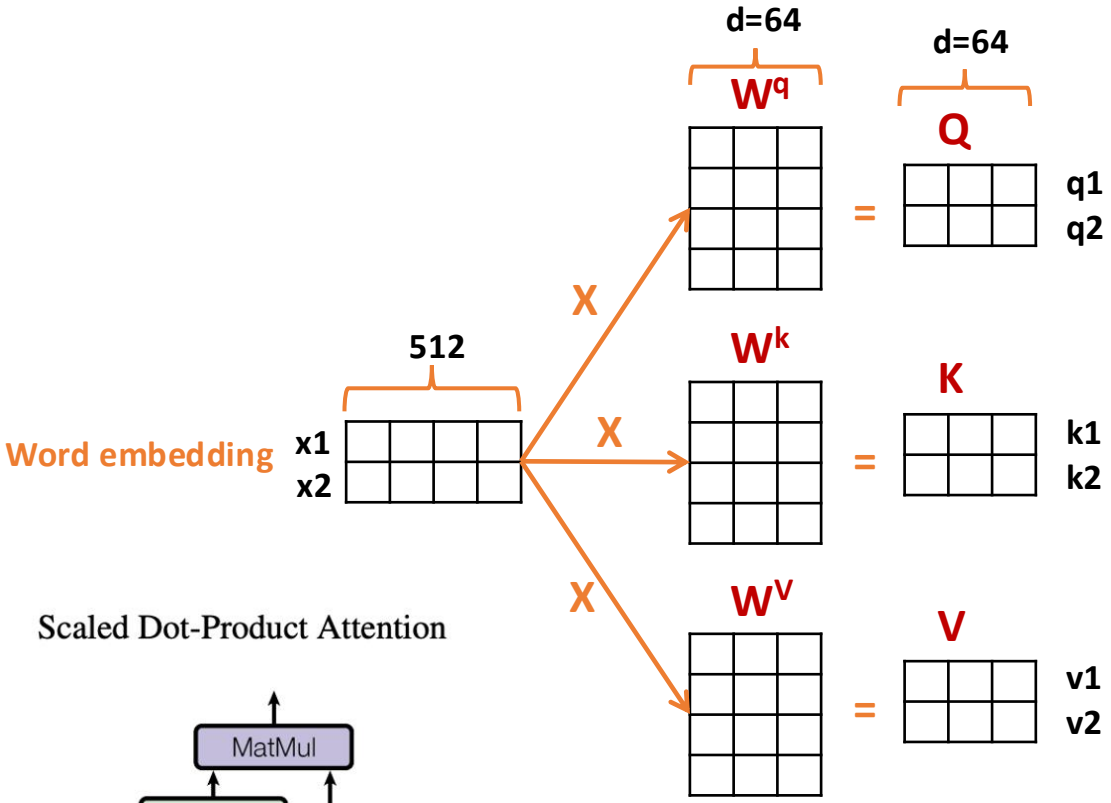




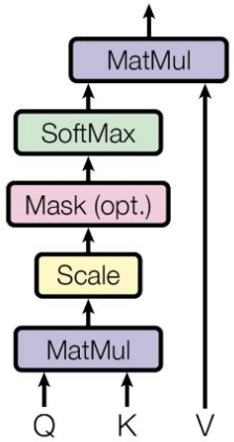
Attention Is All You Need

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathrm{T}}}{\sqrt{d_k}}\right)V$$



Scaled Dot-Product Attention



Dot Product

Score = Q • K =

q1.k1	q1.k2
q2.k1	q2.k2

$$\frac{}{\sqrt{d_k}}$$

=

DP11	DP12
DP21	DP22

SM = SoftMax (

DP11	DP12
DP21	DP22

) =

0.88	0.12
0.11	0.89

Z = SM . V =

0.88	0.12
0.11	0.89

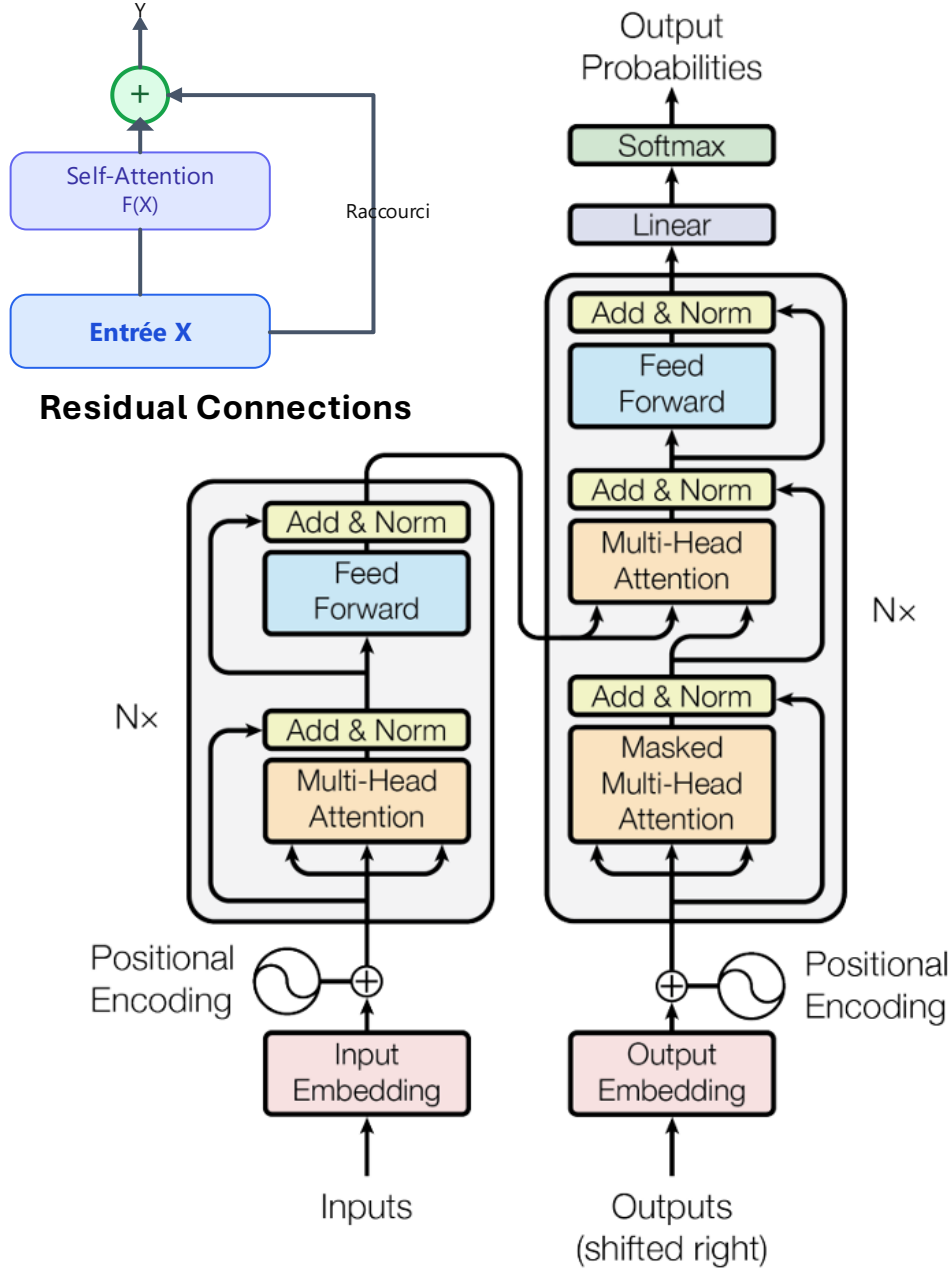
X

=

0.88	0.12
0.11	0.89

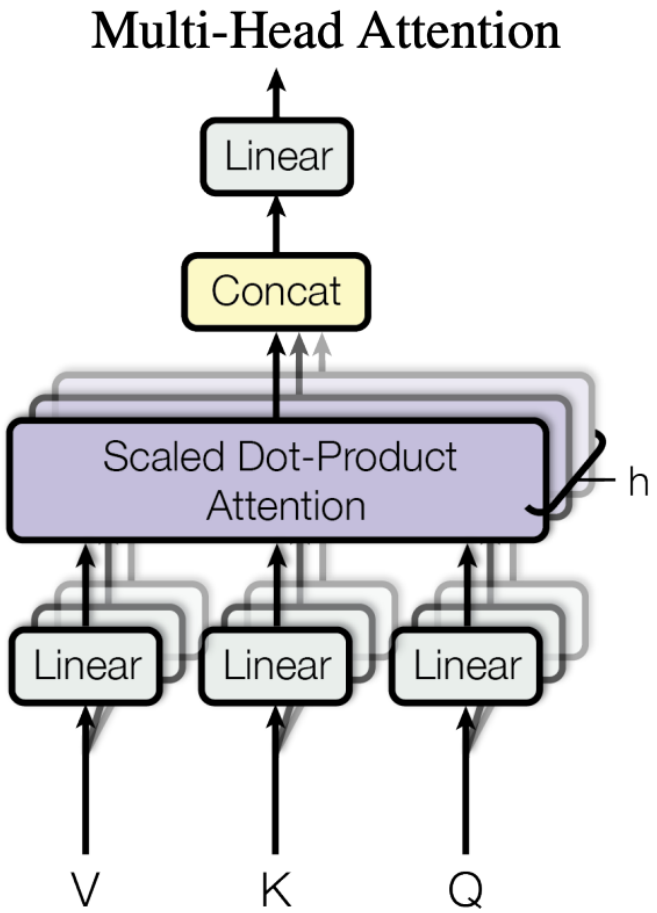
Architecture d'un Transformer : Composants principaux

- Le Transformer original comporte deux grandes parties :
  - **Encoder** (encodeur) : transforme une séquence d'entrée en représentations contextualisées.
  - **Decoder** (décodeur) : génère une séquence de sortie en utilisant les représentations de l'encodeur et les tokens déjà produits.
- Composant principaux de cette architecture :
  - **Token Embedding**: Transformer les identifiants des tokens en vecteurs
  - **Positional Encoding**: Introduire l'information de position
  - **Self-Attention**: Calculer les relations entre les tokens
  - **Multi-Head Attention**: Apprendre plusieurs types de relations
  - **Feed Forward Network**: Transformer individuellement les représentations
  - **Residual Connections**: Faciliter la propagation des informations et des gradients
  - **Layer Normalization**: Stabiliser l'entraînement
  - **Cross-Attention**: Permettre au décodeur de consulter l'encodeur



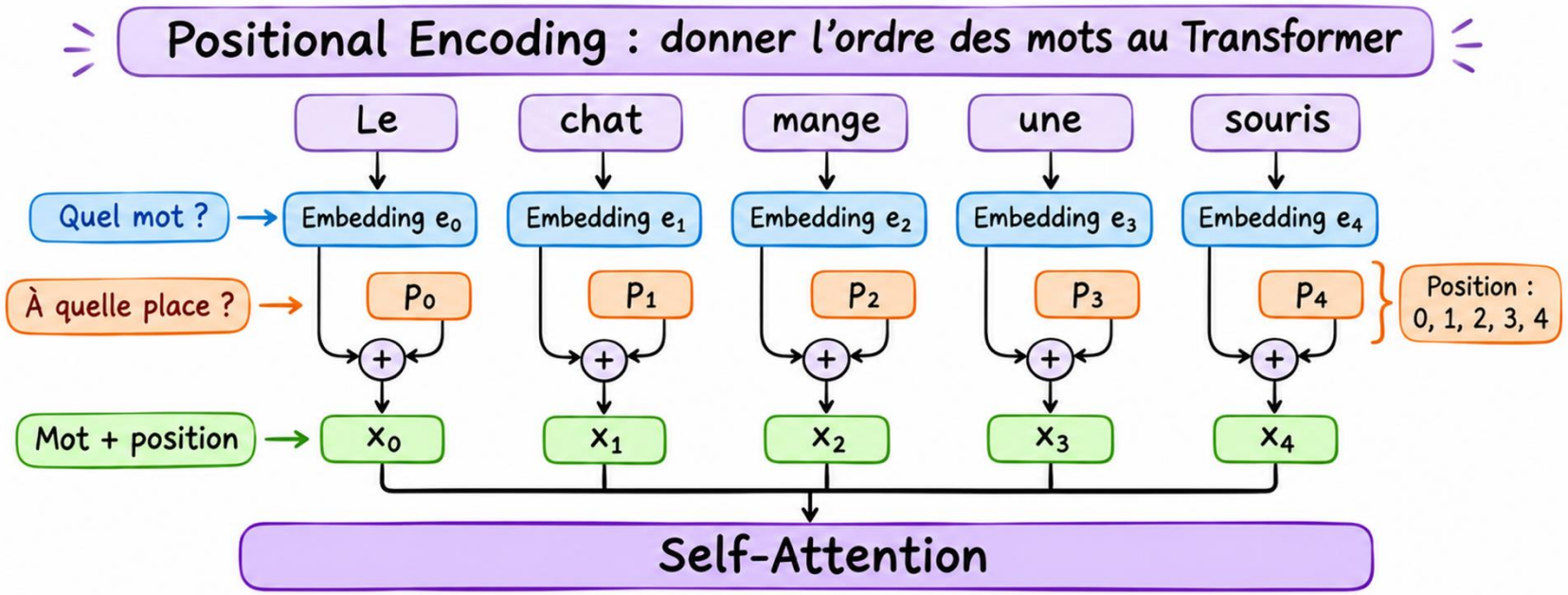
Multi Head Attention

- Au lieu d'effectuer une seule attention, le Transformer en effectue plusieurs en parallèle.
- Par exemple, avec quatre têtes :
  - Une tête peut apprendre des **relations grammaticales**.
  - Une autre peut apprendre des **relations sémantiques**.
  - Une troisième peut se concentrer sur les **mots voisins**.
  - Une quatrième peut détecter des **dépendances éloignées**.
- Ces spécialisations ne sont pas programmées explicitement : elles peuvent émerger pendant l'entraînement.
- Les résultats sont concaténés, puis projetés vers la dimension du modèle.



Positional encoding

- L'attention seule ne connaît pas l'ordre des mots.
- Il faut donc ajouter une information de position aux embeddings.
- Dans le Transformer original, cette information utilise des fonctions sinus et cosinus :



$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

- Les dimensions sont regroupées **deux par deux** : (0,1),(2,3),(4,5),…
- Pour chaque paire, on utilise :
  - **sinus** pour la dimension paire (2i)
  - **cosinus** pour la dimension impaire (2i+1)
- Donc :

$$PE(pos, 0) = \sin\left(\frac{pos}{10000^{0}}\right) \qquad PE(pos, 2) = \sin\left(\frac{pos}{10000^{2/d}}\right)$$

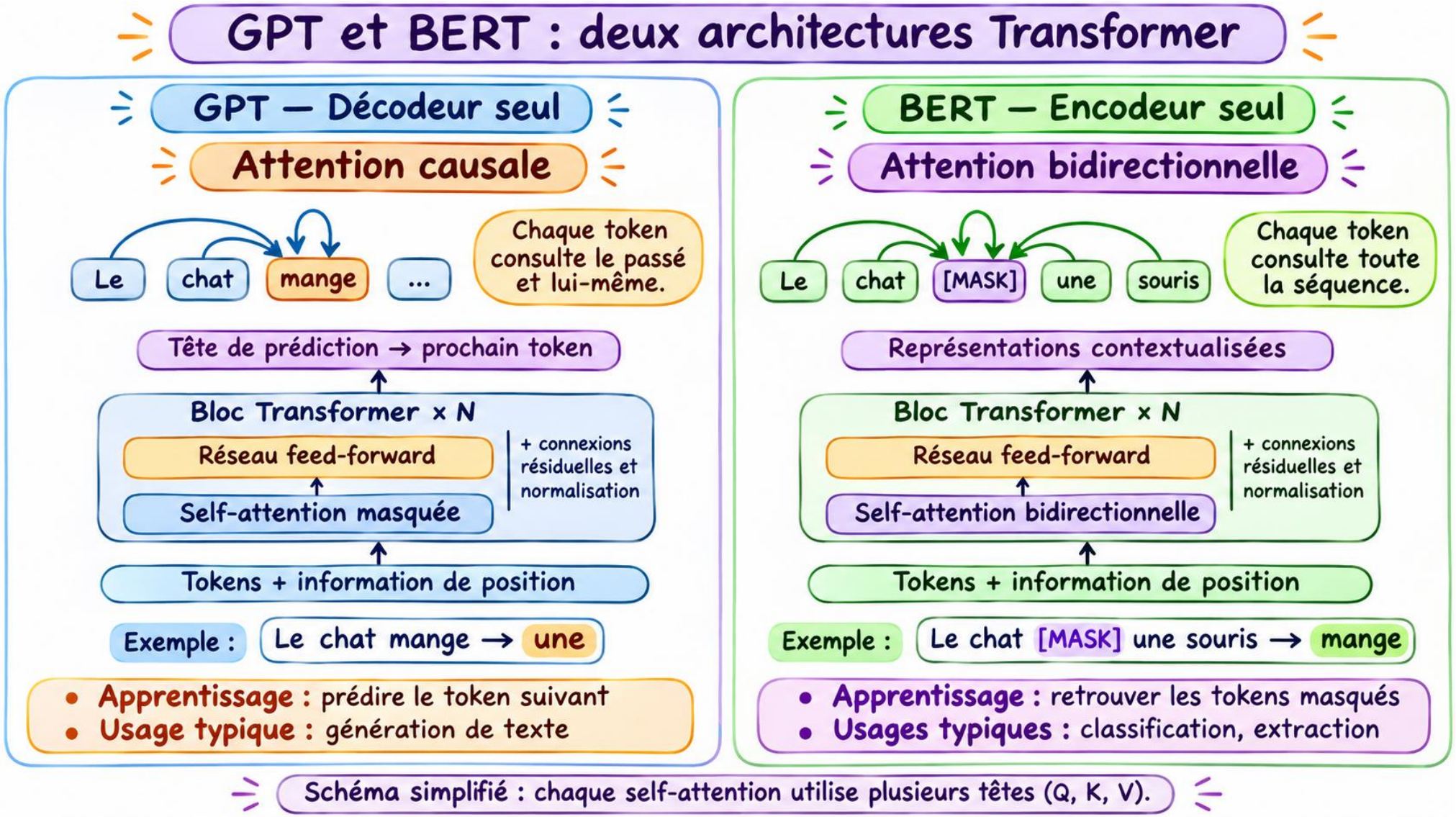
$$PE(pos, 1) = \cos\left(\frac{pos}{10000^{0}}\right) \qquad PE(pos, 3) = \cos\left(\frac{pos}{10000^{2/d}}\right)$$

- GPT et BERT utilisent, dans leurs versions initiales, des embeddings de position appris.

Modèles pré entrainés GPT et BERT

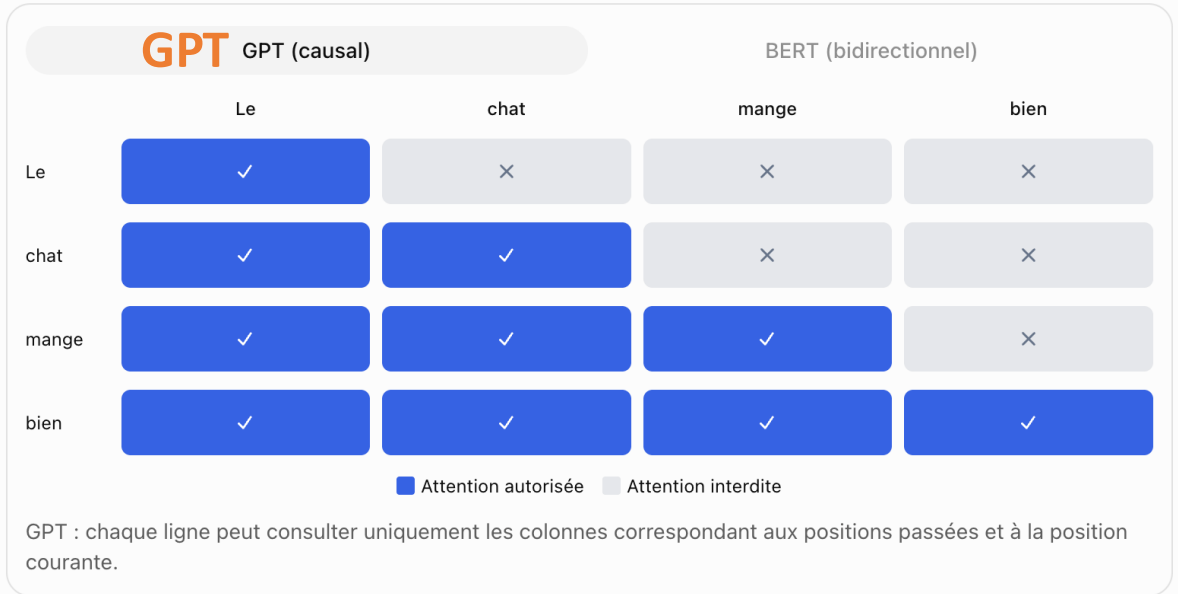
GPT (Generative Pre-trained Transformer). **BERT (Bidirectional Encoder Representations from Transformers).**

- **GPT** apprend à prédire la suite d'une séquence en ne regardant que ce qui précède.
- **BERT** apprend à comprendre un token en regardant son contexte à gauche et à droite.



Comparaison des deux architectures GPT et BERT

Critère	GPT	BERT
Architecture	Decoder-only	Encoder-only
Attention	Causale	Bidirectionnelle
Contexte	Tokens précédents	Tokens précédents et suivants
Apprentissage principal	Prédiction du token suivant	Prédiction de tokens masqués
Sortie habituelle	Distribution sur le prochain token	Représentations de tous les tokens
Applications	Génération, dialogue, code	Classification, extraction, compréhension



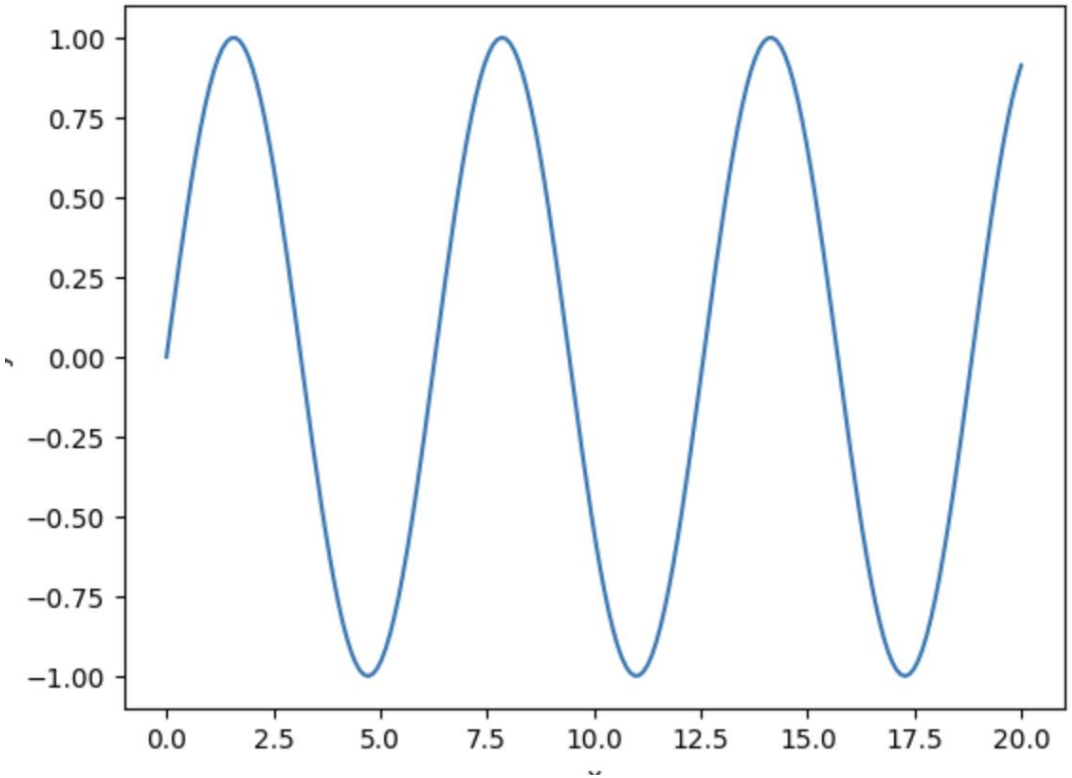


Implémentation avec PyTorch

Données

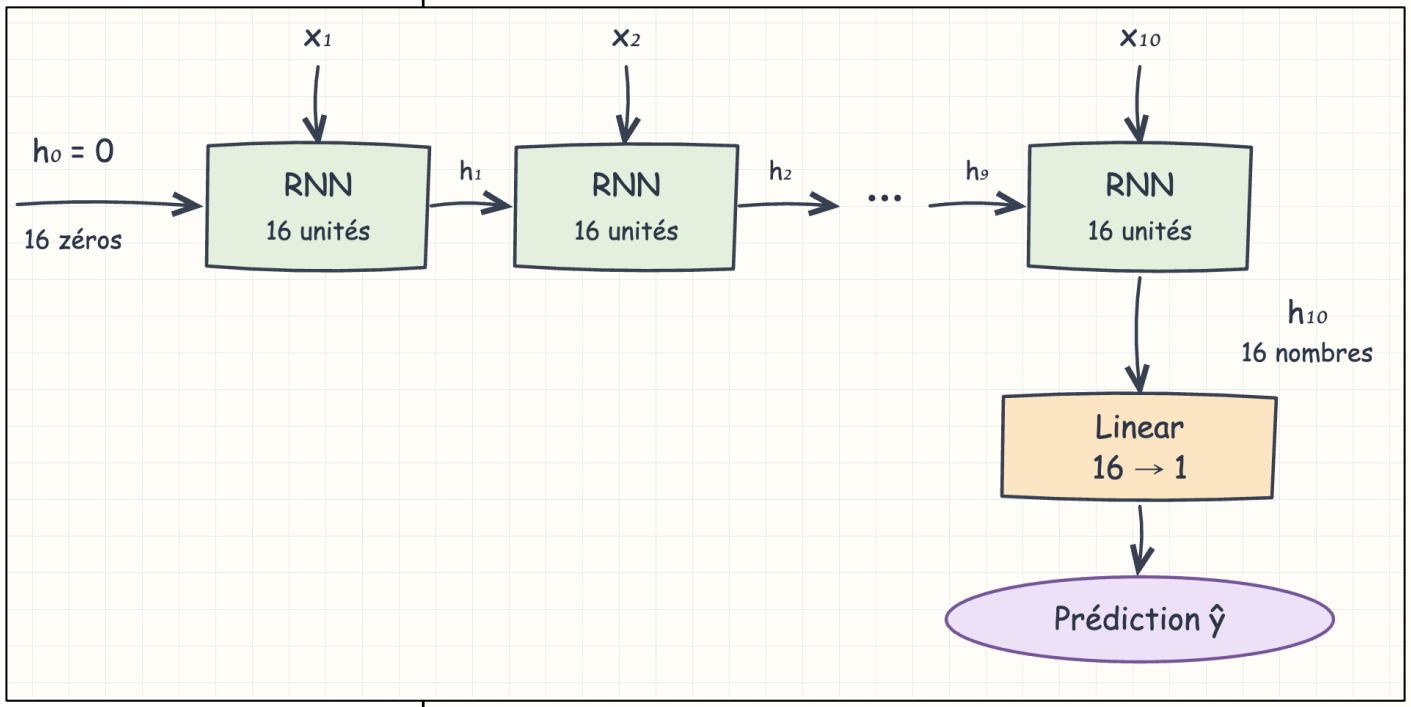
```
import torch
import torch.nn as nn
torch.manual_seed(42)
periode = torch.linspace(0, 20, 200)
serie = torch.sin(periode)
longueur_sequence = 10
X = torch.stack([
    serie[i:i + longueur_sequence]
    for i in range(len(serie) - longueur_sequence)
])
y = torch.stack([
    serie[i + longueur_sequence]
    for i in range(len(serie) - longueur_sequence)
])
X = X.unsqueeze(-1)  # (190, 10, 1)
y = y.unsqueeze(-1)  # (190, 1)
X_train, y_train = X[:150], y[:150]
X_test, y_test = X[150:], y[150:]
```

```
import matplotlib.pyplot as plt
plt.plot(periode, serie)
plt.xlabel("x")
plt.ylabel("y")
```



Implémentation RNN avec PyTorch

```python
class SimpleRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=1,
            hidden_size=16,
            batch_first=True
        )
        self.sortie = nn.Linear(16, 1)
    def forward(self, x):
        etats, _ = self.rnn(x)
        dernier_etat = etats[:, -1, :]
        return self.sortie(dernier_etat)
modele = SimpleRNN()
critere = nn.MSELoss()
optimiseur = torch.optim.Adam(modele.parameters(), lr=0.01)
```



Implémentation RNN avec PyTorch

Entrainement du modèle

```
modele.train()
for epoch in range(300):
    predictions = modele(X_train)
    perte = critere(predictions, y_train)
    optimiseur.zero_grad()
    perte.backward()
    optimiseur.step()
    if (epoch + 1) % 50 == 0:
        print(f"Epoch {epoch + 1:3d} | Erreur : {perte.item():.6f}")

modele.eval()
with torch.no_grad():
    predictions = modele(X_test)
    erreur_test = critere(predictions, y_test)

print(f"\nErreur de test : {erreur_test.item():.6f}")
print(f"Valeur réelle  : {y_test[0].item():.4f}")
print(f"Valeur prédite : {predictions[0].item():.4f}")
```

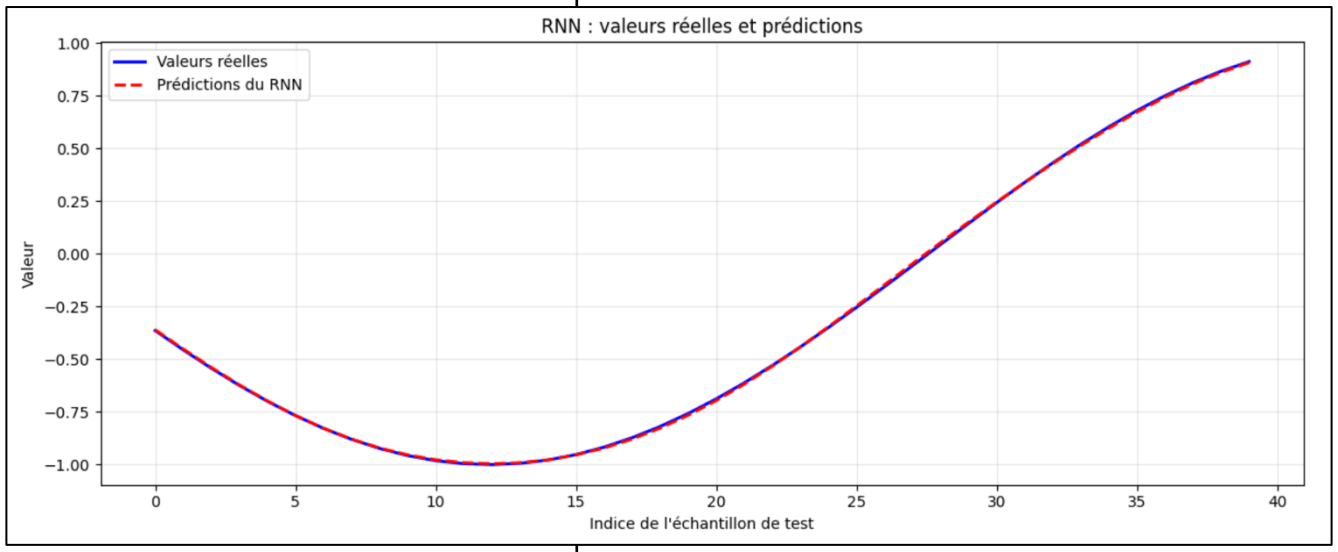
```
Epoch  50 | Erreur : 0.001277
Epoch 100 | Erreur : 0.000504
Epoch 150 | Erreur : 0.000223
Epoch 200 | Erreur : 0.000103
Epoch 250 | Erreur : 0.000049
Epoch 300 | Erreur : 0.000025

Erreur de test : 0.000028
Valeur réelle  : -0.3639
Valeur prédite : -0.3601
```

Implémentation RNN avec PyTorch

Plot

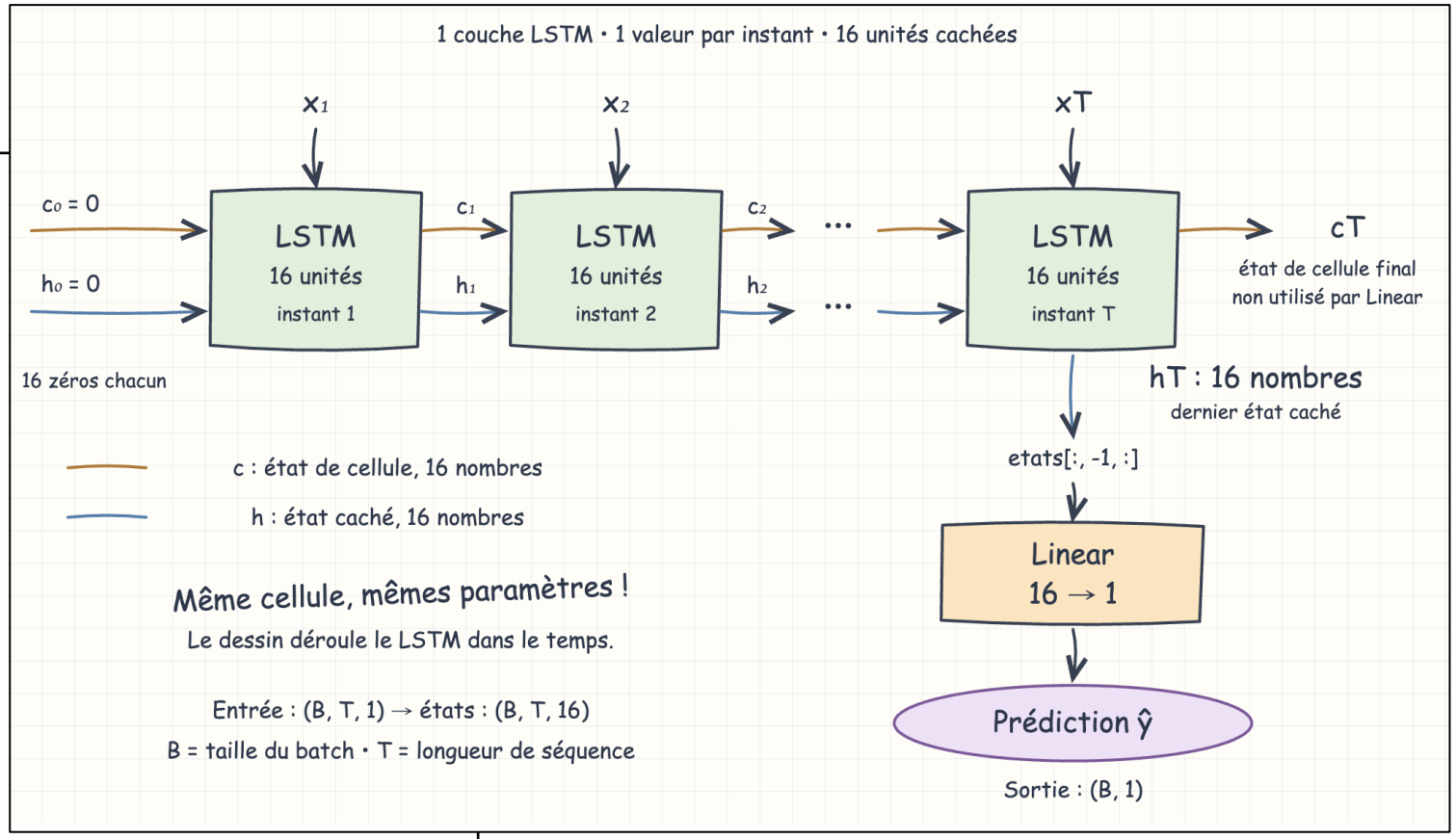
```
import matplotlib.pyplot as plt
# Convertir les tenseurs en tableaux NumPy
valeurs_reelles = y_test.detach().cpu().numpy().reshape(-1)
valeurs_predites = predictions.detach().cpu().numpy().reshape(-1)
# Tracer les deux courbes
plt.figure(figsize=(12, 5))
plt.plot( valeurs_reelles, label="Valeurs réelles",color="blue",linewidth=2)
plt.plot(valeurs_predites,label="Prédictions du RNN",color="red",
    linestyle="--",
    linewidth=2
)
plt.title("RNN : valeurs réelles et prédictions")
plt.xlabel("Indice de l'échantillon de test")
plt.ylabel("Valeur")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Implémentation LSTM avec PyTorch

LSTM

```
class SimpleLSTM(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=1,
            hidden_size=16,
            num_layers=1,
            batch_first=True
        )
        self.sortie = nn.Linear(16, 1)
    def forward(self, x):
        etats, (h_final, c_final) = self.lstm(x)
        dernier_etat = etats[:, -1, :]
        return self.sortie(dernier_etat)
modele = SimpleLSTM()
critere = nn.MSELoss()
optimiseur = torch.optim.Adam(modele.parameters(), lr=0.01)
```



Implémentation LSTM avec PyTorch

Entrainement du modèle

```
modele.train()
for epoch in range(300):
    predictions = modele(X_train)
    perte = critere(predictions, y_train)
    optimiseur.zero_grad()
    perte.backward()
    optimiseur.step()
    if (epoch + 1) % 50 == 0:
        print(f"Epoch {epoch + 1:3d} | Erreur : {perte.item():.6f}")

modele.eval()
with torch.no_grad():
    predictions = modele(X_test)
    erreur_test = critere(predictions, y_test)

print(f"\nErreur de test : {erreur_test.item():.6f}")
print(f"Valeur réelle  : {y_test[0].item():.4f}")
print(f"Valeur prédite : {predictions[0].item():.4f}")
```

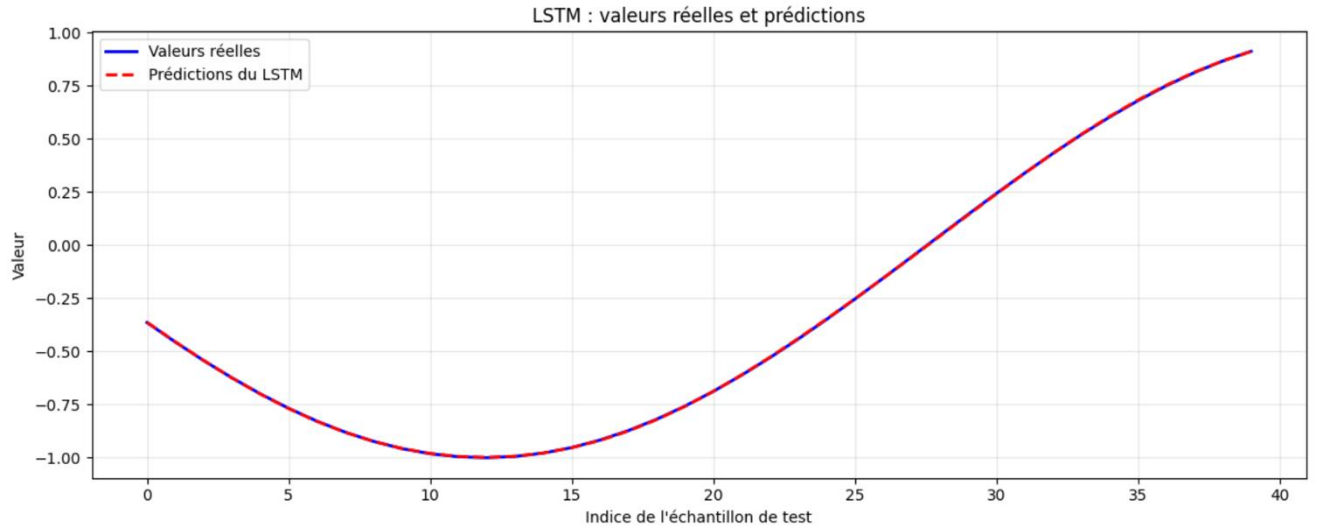
```
Epoch  50 | Erreur : 0.003562
Epoch 100 | Erreur : 0.000248
Epoch 150 | Erreur : 0.000043
Epoch 200 | Erreur : 0.000006
Epoch 250 | Erreur : 0.000004
Epoch 300 | Erreur : 0.000003

Erreur de test : 0.000003
Valeur réelle  : -0.3639
Valeur prédite : -0.3622
```

Implémentation LSTM avec PyTorch

Plot

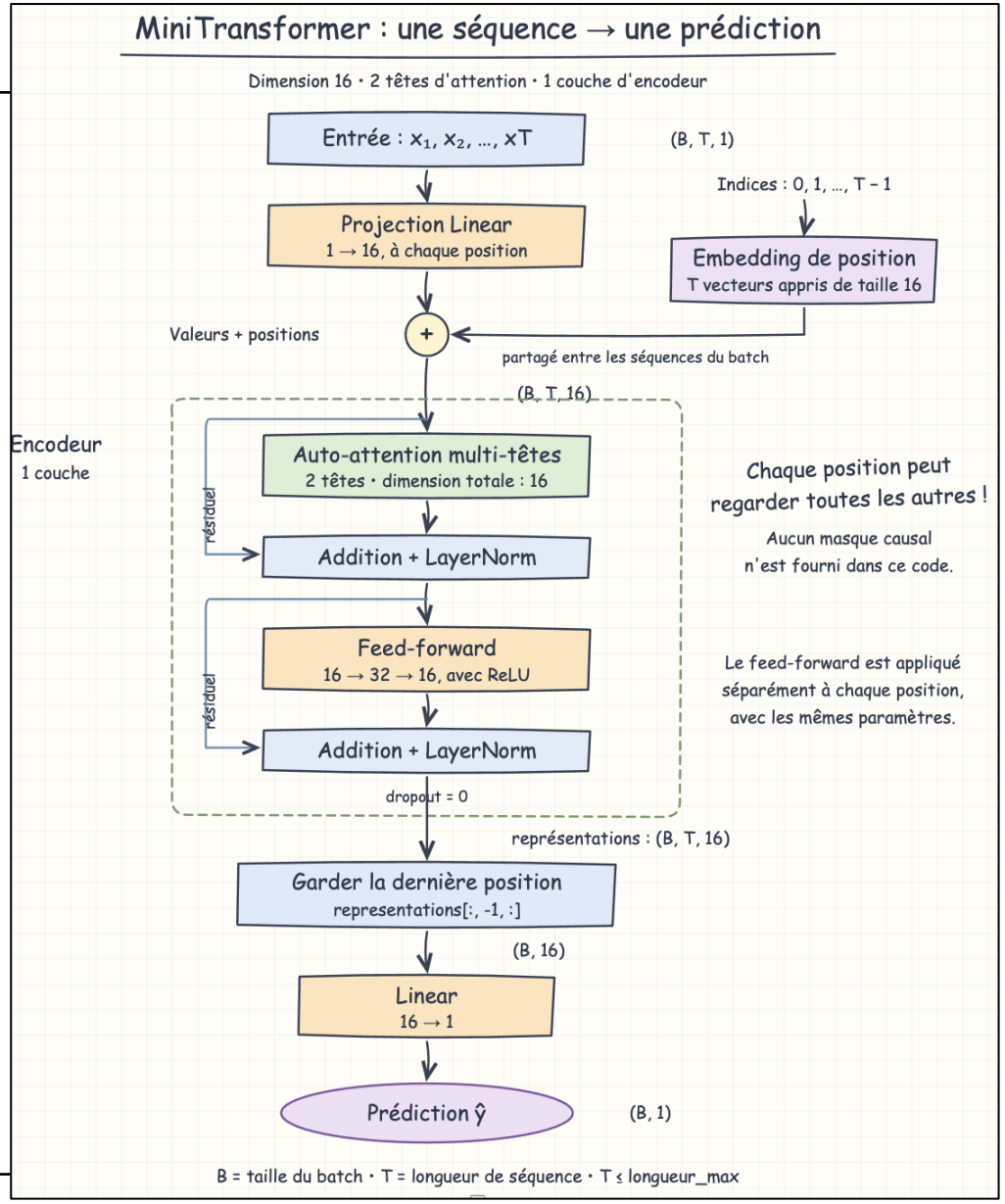
```
import matplotlib.pyplot as plt
# Convertir les tenseurs en tableaux NumPy
valeurs_reelles = y_test.detach().cpu().numpy().reshape(-1)
valeurs_predites = predictions.detach().cpu().numpy().reshape(-1)
# Tracer les deux courbes
plt.figure(figsize=(12, 5))
plt.plot( valeurs_reelles, label="Valeurs réelles",color="blue",linewidth=2)
plt.plot(valeurs_predites,label="Prédictions du LSTM,color="red",
    linestyle="--",
    linewidth=2
)
plt.title( »LSTM : valeurs réelles et prédictions")
plt.xlabel("Indice de l'échantillon de test")
plt.ylabel("Valeur")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Implémentation Transformer avec PyTorch

Modèle

```
class MiniTransformer(nn.Module):
    def __init__(self, longueur_max=10):
        super().__init__()
        dimension = 16
        self.projection = nn.Linear(1, dimension)
        self.positions = nn.Embedding(longueur_max, dimension)
        couche = nn.TransformerEncoderLayer(
            d_model=dimension,  nhead=2, dim_feedforward=32,
            dropout=0.0, batch_first=True
        )
        self.encodeur = nn.TransformerEncoder(couche, num_layers=1)
        self.sortie = nn.Linear(dimension, 1)
    def forward(self, x):
        longueur = x.size(1)
        x = self.projection(x)  # (B, 10, 16)
        indices = torch.arange(longueur, device=x.device)
        positions = self.positions(indices)  # (10, 16)
        x = x + positions.unsqueeze(0)  # (B, 10, 16)
        representations = self.encodeur(x)  # (B, 10, 16)
        derniere_position = representations[:, -1, :]  # (B, 16)
        return self.sortie(derniere_position)  # (B, 1)
modele = MiniTransformer(longueur_max=longueur_sequence)
critere = nn.MSELoss()
optimiseur = torch.optim.Adam(modele.parameters(), lr=0.01)
```



Implémentation LSTM avec PyTorch

Entrainement du modèle

```
modele.train()
for epoch in range(300):
    predictions = modele(X_train)
    perte = critere(predictions, y_train)
    optimiseur.zero_grad()
    perte.backward()
    optimiseur.step()
    if (epoch + 1) % 50 == 0:
        print(f"Epoch {epoch + 1:3d} | Erreur : {perte.item():.6f}")
modele.eval()
with torch.no_grad():
    predictions = modele(X_test)
    erreur_test = critere(predictions, y_test)
print(f"\nErreur de test : {erreur_test.item():.6f}")
print(f"Valeur réelle  : {y_test[0].item():.4f}")
print(f"Valeur prédite : {predictions[0].item():.4f}")
```

```
Epoch  50 | Erreur : 0.001504
Epoch 100 | Erreur : 0.000110
Epoch 150 | Erreur : 0.000026
Epoch 200 | Erreur : 0.000012
Epoch 250 | Erreur : 0.000007
Epoch 300 | Erreur : 0.000004

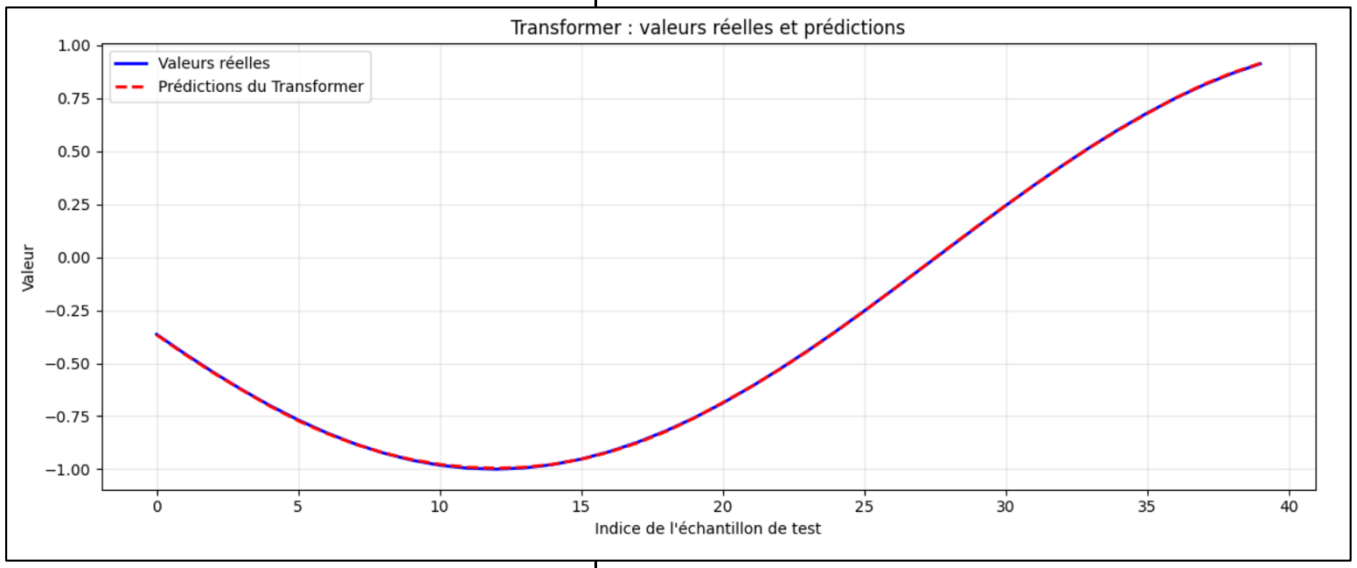
Erreur de test : 0.000006
Valeur réelle  : -0.3639
Valeur prédite : -0.3662
```

Implémentation Transformer avec PyTorch

```
import matplotlib.pyplot as plt
valeurs_reelles = y_test.detach().cpu().numpy().reshape(-1)
valeurs_predites = predictions.detach().cpu().numpy().reshape(-1)
plt.figure(figsize=(12, 5))
plt.plot(valeurs_reelles,label="Valeurs réelles",
   color="blue",linewidth=2)
plt.plot(
   valeurs_predites,
   label="Prédictions du Transformer",
   color="red",linestyle="--",linewidth=2
)

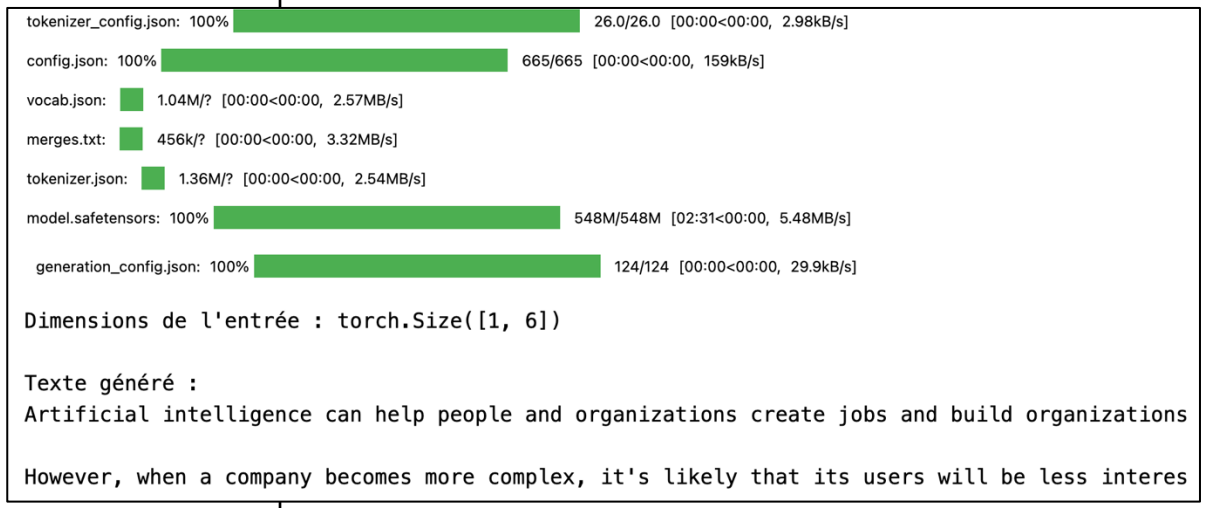
plt.title("Transformer : valeurs réelles et prédictions")
plt.xlabel("Indice de l'échantillon de test")
plt.ylabel("Valeur")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Plot



Implémentation basée sur GPT Transformer avec PyTorch

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
nom_modele = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(nom_modele)
modele = AutoModelForCausalLM.from_pretrained(nom_modele)
tokenizer.pad_token = tokenizer.eos_token
modele.config.pad_token_id = tokenizer.pad_token_id
modele.eval()
texte = "Artificial intelligence can help people"
entrees = tokenizer( texte, return_tensors="pt » )
print("Dimensions de l'entrée :", entrees["input_ids"].shape)
with torch.no_grad():
   sortie = modele.generate(
      **entrees, max_new_tokens=50,
      do_sample=True, temperature=0.8,top_p=0.9
      pad_token_id=tokenizer.pad_token_id )
texte_genere = tokenizer.decode(sortie[0], skip_special_tokens=True)
print("\nTexte généré :")
print(texte_genere)
```



Implémentation basée sur BERT Transformer avec PyTorch

```python
import torch
from transformers import AutoTokenizer, AutoModelForMaskedLM
nom_modele = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(nom_modele)
modele = AutoModelForMaskedLM.from_pretrained(nom_modele)
modele.eval()
texte = f"Paris est la {tokenizer.mask_token} de la France."
entrees = tokenizer( texte, return_tensors="pt")
print("Texte :", texte)
print("Dimensions de l'entrée :", entrees["input_ids"].shape)
position_masque = (
    entrees["input_ids"][0] == tokenizer.mask_token_id
).nonzero(as_tuple=True)[0].item()
with torch.no_grad():
    sortie = modele(**entrees)
scores = sortie.logits[0, position_masque, :]
probabilites = torch.softmax(scores, dim=-1)
top_probabilites, top_indices = torch.topk(probabilites, k=5)
print("\nPropositions de BERT :")
for probabilite, indice in zip(top_probabilites, top_indices):
    token = tokenizer.decode([indice.item()]).strip()
    phrase = texte.replace(tokenizer.mask_token, token)
    print(f"{probabilite.item():.2%} | {phrase}")
```

```
tokenizer_config.json: 100%  49.0/49.0 [00:00<00:00, 24.0kB/s]
config.json: 100%  625/625 [00:00<00:00, 399kB/s]
vocab.txt:  996k/? [00:00<00:00, 5.26MB/s]
tokenizer.json:  1.96M/? [00:00<00:00, 4.76MB/s]
model.safetensors: 100%  714M/714M [03:18<00:00, 6.15MB/s]
Some weights of the model checkpoint at bert-base-multilingual-cased were not used when initi
- This IS expected if you are initializing BertForMaskedLM from the checkpoint of a model tra
- This IS NOT expected if you are initializing BertForMaskedLM from the checkpoint of a model
Texte : Paris est la [MASK] de la France.
Dimensions de l'entrée : torch.Size([1, 10])

Propositions de BERT :
36.33% | Paris est la capitale de la France.
21.09% | Paris est la ville de la France.
3.16% | Paris est la Cité de la France.
1.67% | Paris est la préfecture de la France.
1.64% | Paris est la banlieue de la France.
```

Activité Pratique N°4

Architecture Transformer

Travail à faire (Voir classroom : Note Book et PDF

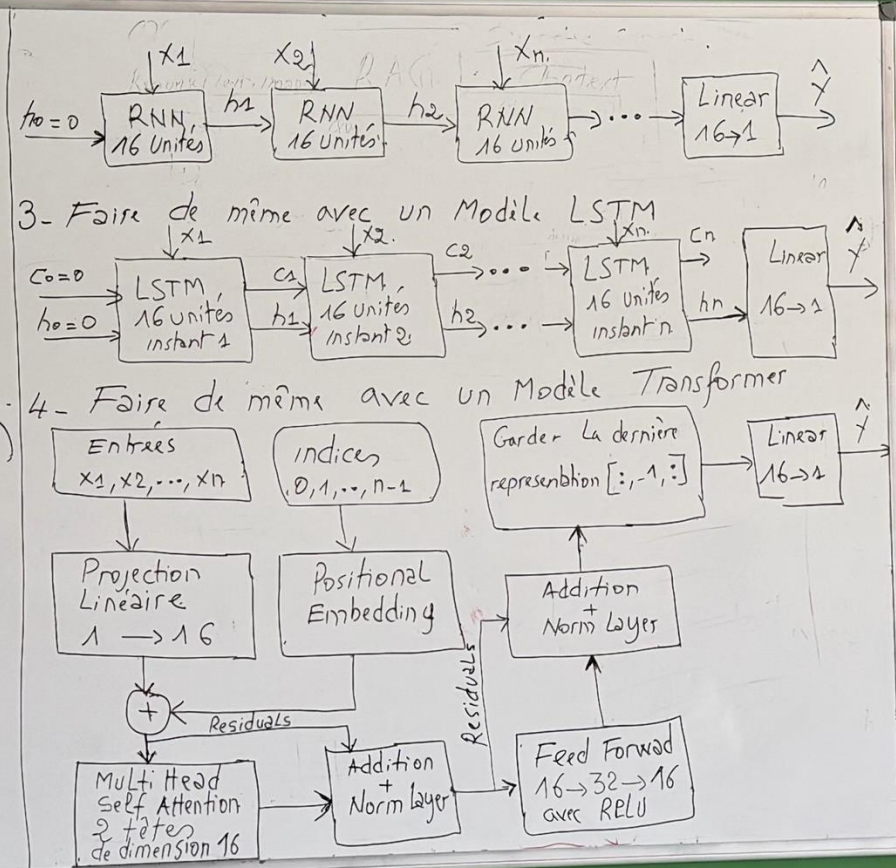
Créer un Note Book qui contient les implémentation des parties suivantes :

1- Générer un Data set sous forme d'une série Temporelle.
   $s = \sin(x)$ (Prendre 200 valeurs entre 0 et 20)
   - Ranger le dataset sous formes de petites séquences de dimension 10

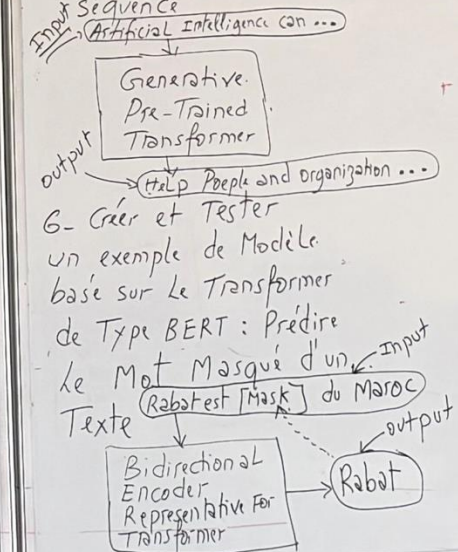
$$X = \left[[s_0, s_1, s_3, \dots, s_9], [s_1, s_2, s_3, \dots, s_{10}], [s_2, s_3, s_4, \dots, s_{11}], \dots \right]$$

$$Y = \left[[s_{10}], [s_{11}], [s_{12}], \dots \right]$$

2- Créer, Entrainer, Tester un Modèle basé sur RNN qui permet de prédire la valeur suivate d'un séquence X test en utilisant PyTorch



5- Créer et Tester un Exemple de Modèle basé sur le Transformer de Type GPT (Prédire le Texte suivant d'une séquence



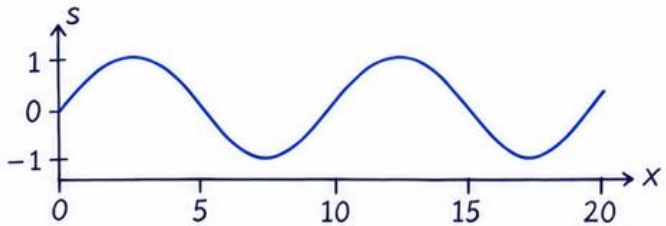
6- Créer et Tester un exemple de Modèle basé sur Le Transformer de Type BERT : Prédire le Mot Masqué d'un Texte

Activité pratique n° 4 — Architecture Transformer

Travail à faire : créer un notebook PyTorch (voir Classroom : notebook et PDF)

1. Préparer une série temporelle

- $s = \sin(x)$ • 200 valeurs de x entre 0 et 20
- Former des séquences de longueur 10



X : $[s_0, \ldots, s_9]$ ⟶ y : s_{10}

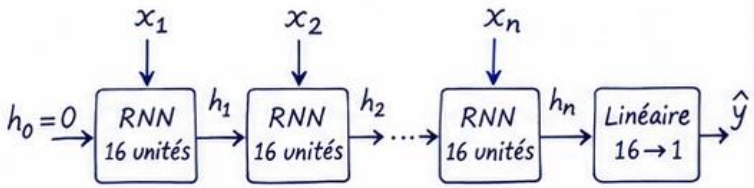
X : $[s_1, \ldots, s_{10}]$ ⟶ y : s_{11}

X : $[s_2, \ldots, s_{11}]$ ⟶ y : s_{12}

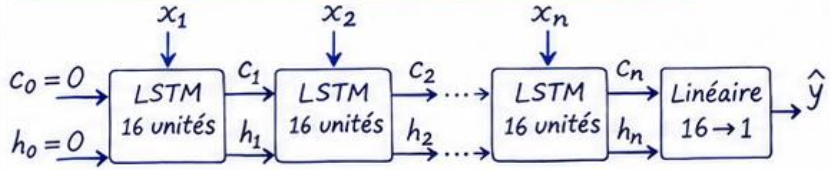
...

2. Créer, entraîner et tester un RNN

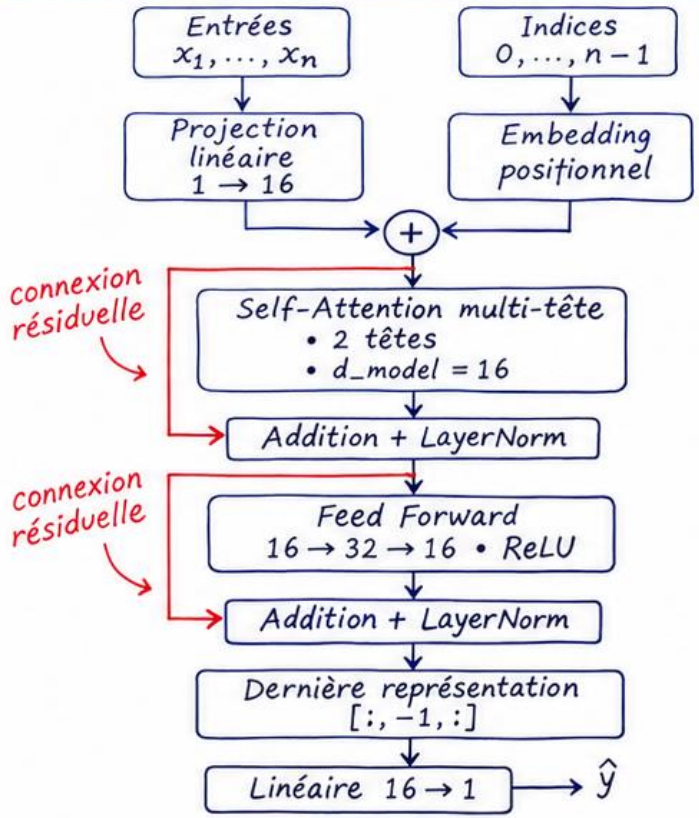
- Prédire la valeur suivante d'une séquence X_{test}.



3. Faire de même avec un LSTM

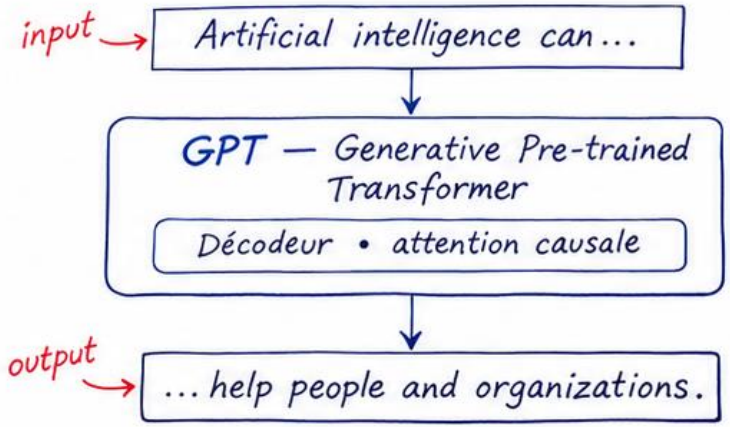


4. Faire de même avec un Transformer



5. Créer et tester un modèle GPT

- Prédire la suite d'un texte



6. Créer et tester un modèle BERT

- Prédire un mot masqué

